```
  for k in {1..maxSteps} do
                                      #2
    T ← temperature(T0, k,
maxSteps)                                                  #3
    P ← randomStep(P0.clone(), T)
                  #4
    if acceptance(C(P), C(P0), T) > randomFloat(0,1)
then                        #5
      P0 ← P
  return P0
```

#1 Method `simulatedAnnealing` takes a cost function `C`, a starting point `P0`, a starting temperature `T0`, a probability function `acceptance` and the maximum number of iterations to perform, `maxSteps`. It returns a point in the domain: ideally the point where `C` has the smallest value (either locally or globally).

#2 Starts the iteration, running the main cycle at most `maxSteps` times.

#3 Set the temperature of the system, based on the initial temperature and current iteration.

#4 Creates a new point `P` in the domain, to which the system should transition.

#5 If the probability of transitioning from `P0` to `P` is higher than a random floating point number, drawn between `0` and `1`, then update current state to `P`.

17.1.3 Why Simulated Annealing Works

If you feel the way simulated annealing works is “slightly” counterintuitive, well you are not alone: after all, if the